

Mise à jour mobile — septembre 2026

Ce qui a changé en production depuis la livraison des abonnements, et ce que l'application doit faire. Les exemples sont des réponses **réelles**, relevées sur la production.

Complète, sans les remplacer : [abonnements](#) · [quotas gratuits](#) · [codes de réduction](#) · [complétion du profil](#)

0. En un coup d'œil

changement	urgence	en production ?
Le verrou est ACTIF — les 403 ne sont plus simulés	🔴 bloquant	oui , depuis le 11/09
Rafraîchir le jeton après un achat, sinon l'IA reste plafonnée	🔴 bloquant	pas encore — §2
<code>POST /codes/valider</code> ne demande plus de compte	🟡 opportunité	oui
Les plans portent une liste d' <code>avantages</code> à afficher	🟡 à intégrer	oui
La complétion du profil compte 15 champs, l'email a fusionné	🟢 informatif	oui

1. 🟡 Le verrou est actif : les refus sont réels

C'est le changement le plus important. Jusqu'ici, le serveur **calculait** les refus et servait quand même la ressource ; toutes les documentations précédentes disaient « le verrou est éteint au lancement ».

Ce n'est plus vrai. Depuis le 11/09, un utilisateur qui dépasse son quota reçoit un vrai 403 :

```
GET /epreuves/2036/telechargement

{
  "statusCode": 403,
  "error": "QUOTA_EXCEEDED",
  "message": "Vous avez consulté vos 5 ressources gratuites. Un abonnement est requis pour continuer.",
  "feature": "EPREUVE_VIEW",
  "quota": { "used": 5, "limit": 5 }
}
```

Ce qui est effectivement appliqué aujourd'hui

situation	réponse
au-delà de 5 ressources académiques distinctes par mois	<code>403 QUOTA_EXCEEDED</code>
concours sans abonnement actif	<code>403 SUBSCRIPTION_REQUIRED</code>
au-delà de 1 lancement Ketsia par mois	<code>403 QUOTA_EXCEEDED</code>
profil sous le seuil	<code>403 PROFIL_INCOMPLET</code> — pas encore appliqué , le seuil reste inactif

Les **administrateurs ne sont jamais concernés** : si vous testez avec un compte admin, vous ne verrez jamais de refus. C'est la première cause de « chez moi ça marche » sur ce sujet — **testez avec un compte étudiant**.

Deux pièges de comptage

Le quota compte des ressources DISTINCTES. Rouvrir la même épreuve une dixième fois ne consomme rien. N'affichez donc pas un compteur qui s'incrémente à chaque ouverture : lisez `quota.used` renvoyé par le serveur.

Au-delà du plafond, plus rien n'est enregistré. Le compteur reste figé à `5/5` quel que soit le nombre de tentatives. Un écran qui afficherait « 8 tentatives sur 5 » serait faux.

`mes-droits` **reste la source à interroger**

```
GET /abonnements/mes-droits
```

```
{
  "verrou_actif": true,
  "droits": {
    "CONCOURS_DOWNLOAD": { "allowed": false, "reason": "SUBSCRIPTION_REQUIRED" },
    "EPREUVE_VIEW": { "allowed": false, "reason": "QUOTA_EXCEEDED", "quota": { "used": 5, "limit": 5 } },
    "EXAMEN_NAT_VIEW": { "allowed": false, "reason": "QUOTA_EXCEEDED", "quota": { "used": 5, "limit": 5 } },
    "KETSIA_AI": { "allowed": true, "reason": "FREE_QUOTA" }
  }
}
```

`verrou_actif` vaut désormais `true`. Tant qu'il valait `false`, un `allowed: false` était théorique ; il ne l'est plus. Continuez à griser les boutons à partir de `allowed`, mais sachez que l'utilisateur se heurtera réellement au refus s'il force.

Aiguillez toujours sur `error`, jamais sur le code HTTP. Trois refus partagent le 403 et appellent trois écrans différents — voir §5.

2. 🚫 Rafraîchir le jeton après un achat

⚠️ **Pas encore déployé.** Cette section décrit un changement en attente de revue. Le claim n'existe pas encore dans les jetons de production : un décodage aujourd'hui ne le trouvera pas. Elle est écrite à l'avance parce que le travail côté mobile — rafraîchir après un achat — est indépendant et peut être préparé dès maintenant. Nous préviendrons à la mise en production.

Le jeton d'accès portera un claim `abonnement_actif`, que le backend IA (Ketsia) lit pour décider s'il plafonne l'usage de l'assistante.

```
{ "sub": 26785, "email": "...", "role": "étudiant", "abonnement_actif": true }
```

C'est une photographie prise à l'émission du jeton, et le jeton vit 24 h.

Conséquence directe, et c'est le piège : un utilisateur qui vient de payer garde `abonnement_actif: false`, donc reste plafonné sur Ketsia jusqu'à l'expiration de son jeton — jusqu'à une journée après avoir payé.

Ce que l'application doit faire

Immédiatement après une souscription réussie, appeler :

```
POST /auth/refresh
{ "refresh_token": "..." }
```

et remplacer le jeton stocké par celui renvoyé. Le claim y sera à `true`.

Vérifié sur l'environnement de développement :

sans abonnement	<code>abonnement_actif: false</code>
abonnement actif, login	<code>abonnement_actif: true</code>
abonnement actif, refresh	<code>abonnement_actif: true</code>
abonnement échoué, login	<code>abonnement_actif: false</code>

Ne lisez pas ce claim pour décider quoi afficher. Il peut avoir 24 h de retard. Pour l'état d'abonnement dans l'interface, utilisez `GET /abonnements/mon-abonnement` ou `mes-droits`, qui interrogent la base. Le claim n'existe que pour Ketsia.

3. 🟡 Valider un code sans compte

`POST /codes/valider` **n'exige plus d'être connecté.** Un code promotionnel peut donc être saisi et vérifié sur un écran d'offre, avant l'inscription.

```
POST /codes/valider
{ "code": "RENTREE2026", "plan_uuid": "..." }
```

```
{
  "valide": true,
  "code": { "code": "RENTREE2026", "libelle": "Rentrée" },
  "effets": {
    "remise": { "type": "POURCENTAGE", "valeur": 20,
      "montant_remise": 400, "prix_initial": 2000, "prix_final": 1600 }
  }
}
```

Le jeton reste **accepté et utile** s'il est présent : envoyez-le quand l'utilisateur est connecté.

Ce qu'un appel anonyme ne peut pas détecter

Deux refus dépendent de l'identité de l'appelant et ne sortiront donc jamais sans jeton :

- `AUTO_UTILISATION` — l'utilisateur essaie son propre code de parrainage ;
- `DEJA_UTILISE` — il a déjà consommé ce code.

L'aperçu peut donc être optimiste. Ne présentez pas le résultat comme un engagement : le code est revalidé sous verrou au moment de la souscription, qui exige un compte, et peut y être refusé. Formulez « code accepté » plutôt que « remise acquise ».

Limitation de débit

Sans compte, le décompte se fait **par adresse IP** : environ 8 essais avant `429` . Sur un réseau partagé — un cybercafé, un campus — le plafond est atteint plus vite. Traitez le `429` en invitant à réessayer dans un instant, pas en affichant une erreur technique.

4. 🟡 Les plans décrivent ce qu'ils apportent

`GET /abonnements/plans` renvoie maintenant une liste `avantages` , une ligne par bénéfice, à afficher en puces sur l'écran d'abonnement.

```
[
  {
    "libelle": "Abonnement mensuel",
    "prix": 2000, "devise": "XOF", "duree_jours": 30,
    "description": "Accès illimité pendant 1 mois",
    "avantages": [
      "Épreuves en illimité",
      "Examens nationaux en illimité",
      "Concours : téléchargement des sujets",
      "Ketsia, l'assistante IA, sans limite"
    ]
  }
]
```

`description` dit la **durée**, `avantages` dit ce qu'on **gagne**. Les deux se complètent.

La liste est **réglable depuis le back-office** : ne la codez pas en dur, ne supposez pas quatre lignes, et gérez le cas `null` ou vide en n'affichant simplement rien. Sa longueur peut changer d'un plan à l'autre.

Aujourd'hui les trois plans annoncent les mêmes avantages — un abonnement annuel ne donne rien de plus, seulement plus longtemps. Ça peut évoluer sans déploiement.

Les avantages apparaissent aussi dans `plan` sur `GET /abonnements/mon-abonnement` , pratique pour un écran « votre abonnement ».

5. L'aiguillage des 403, à revoir

Trois refus partagent le code 403 et appellent trois écrans différents. C'est la règle la plus importante de cette mise à jour, maintenant que les refus sont réels.

error	ce que l'utilisateur doit faire	écran
<code>QUOTA_EXCEEDED</code>	attendre le mois prochain, ou s'abonner	offre d'abonnement
<code>SUBSCRIPTION_REQUIRED</code>	s'abonner	offre d'abonnement
<code>PROFIL_INCOMPLET</code>	compléter son profil	formulaire de profil

Proposer un abonnement sur `PROFIL_INCOMPLET` est le contresens à éviter : **l'abonnement ne débloque rien** dans ce cas — le seuil de profil prime sur l'abonnement.

Ce dernier cas n'est pas encore actif en production, mais le code est en place : préparez l'écran, il basculera par un réglage, sans déploiement.

6. ● La complétion du profil compte 15 champs

GET /utilisateurs/profil/completion est inchangé dans sa forme. Ce qui a changé est ce qui est compté.

L'adresse email ne compte plus pour elle-même. Sans adresse il n'y a pas de compte : la compter créditait tout le monde d'un point acquis d'avance. Elle a fusionné avec sa vérification en un champ unique, `email_verifie`, libellé « Adresse email vérifiée ».

La situation de handicap n'est pas comptée, et ne le sera pas : sa valeur par défaut fait qu'elle n'est jamais vide, elle ne dirait donc rien.

Conséquence : un compte fraîchement inscrit affiche un pourcentage **plus bas qu'avant** pour un profil identique. Rien n'a été perdu, le dénominateur a changé.

```
{
  "pourcentage": 23,
  "champs_total": 13,
  "champs_replis": 3,
  "seuil_requis": 95,
  "seuil_actif": false,
  "conforme": true,
  "manquants": [ { "champ": "telephone", "libelle": "Numéro de téléphone" }, "..." ]
}
```

`champs_total` vaut **13** et non 15 en production : l'administration a retiré `pseudo` et `photo` du calcul. **Ne codez jamais le nombre de champs en dur** — il se règle depuis le back-office. `champs_total` et `manquants` font foi.

Rappel inchangé : testez `conforme`, jamais `pourcentage >= seuil_requis`.

7. Ce qu'il y a à faire, par ordre

1. **Tester avec un compte étudiant**, jamais admin — sinon aucun refus

n'apparaîtra.

1. **Traiter les 403 en aiguillant sur `error`** (§5), pas sur le code HTTP.
2. **Rafraîchir le jeton après chaque souscription réussie** (§2), sans quoi

l'abonné reste plafonné sur Ketsia jusqu'à une journée.

1. Afficher les **avantages** sur l'écran d'abonnement (§4), sans en coder le

nombre.

1. Permettre la **saisie d'un code avant inscription** (§3), en présentant le

résultat comme un aperçu.

1. Vérifier que l'écran de profil lit `champs_total` et non une constante

(§6).